UNIVERSITÀ
DEGLI STUDI
DELL'AQUILA

# Lesson 2 — Data: Exploration and Preparation

Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

2 October 2026

# Before we start

Exercise 1 was due at the start of today’s lesson.

- The wine-quality workflow, end to end
- Marked on **methodology**, not on accuracy
- We will look at a few submissions’ framing choices in a moment

# What today builds on

- Lesson 1’s rule: **nothing is learned before the split**
- Lesson 1’s tool: `Pipeline`, so the rule is structural, not just discipline
- Today: apply both to data that is actually messy

# Today

- Exploratory analysis
- Missing values
- Outliers
- Scaling and encoding
- Feature engineering and pipelines
- Leakage, in a form nothing warns you about

# One dataset, all lesson long

2000 synthetic telecom customers. Will they **churn**?

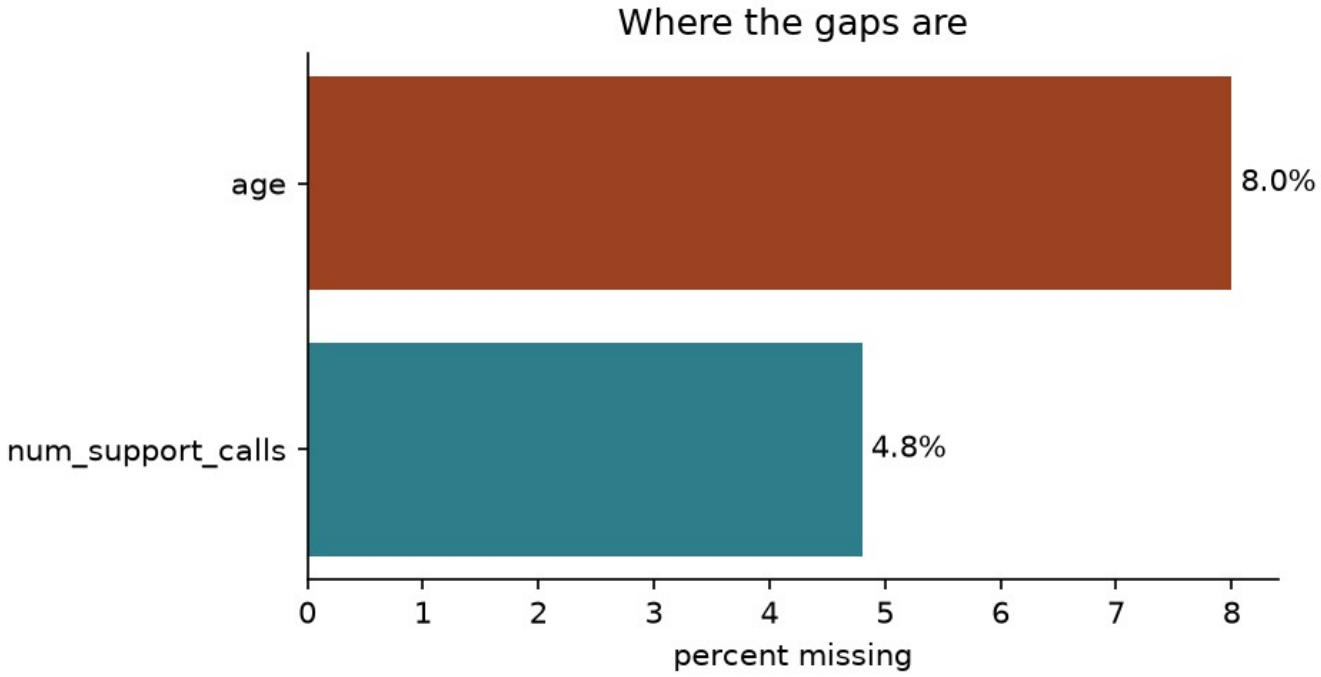
- Numeric: tenure, monthly charges, age, support calls
- Categorical: contract type, region, zip code (493 levels)
- Missing values, outliers, and one column built to carry **no signal**

# Look before you touch anything

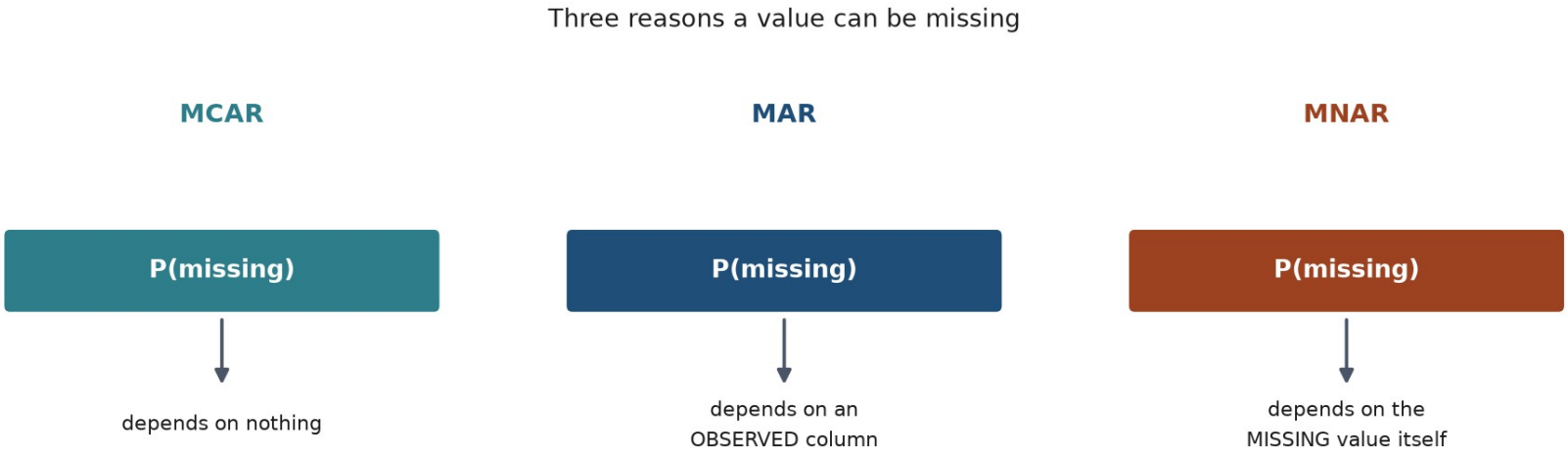
`.info()`, `.describe()`, class balance, first — always.

- 2000 rows, 8 columns, mixed types
- Churn rate: **19.4%** → baseline accuracy 80.6%
- `tenure_months` max: 999. Not a typo in the slide.

# Where the gaps are



# Three reasons a value is missing



# age: MCAR

Some customers just left it blank. Nothing systematic.

- Safe to impute with a fixed statistic
- The bias question is not “is this fair” — it is “what does filling it cost”

# What mean imputation actually costs

Every correlation involving the imputed column shrinks — even under MCAR.

$$\rho(X', Y) = \sqrt{1 - p} \;\; \rho(X, Y)$$

# num_support_calls: MAR

Missing more often for long-tenure customers — not by chance.

- Treating it as MCAR reweights the sample the wrong way
- A **missingness indicator** column preserves the signal even after filling

# What to do about a gap

- **Drop rows** — only if few, and only if MCAR
- **Drop the column** — if it is missing too often to help
- **Impute** — mean/median, most-frequent, `KNNImputer`
- **Add a missingness indicator** — keeps MAR signal alive

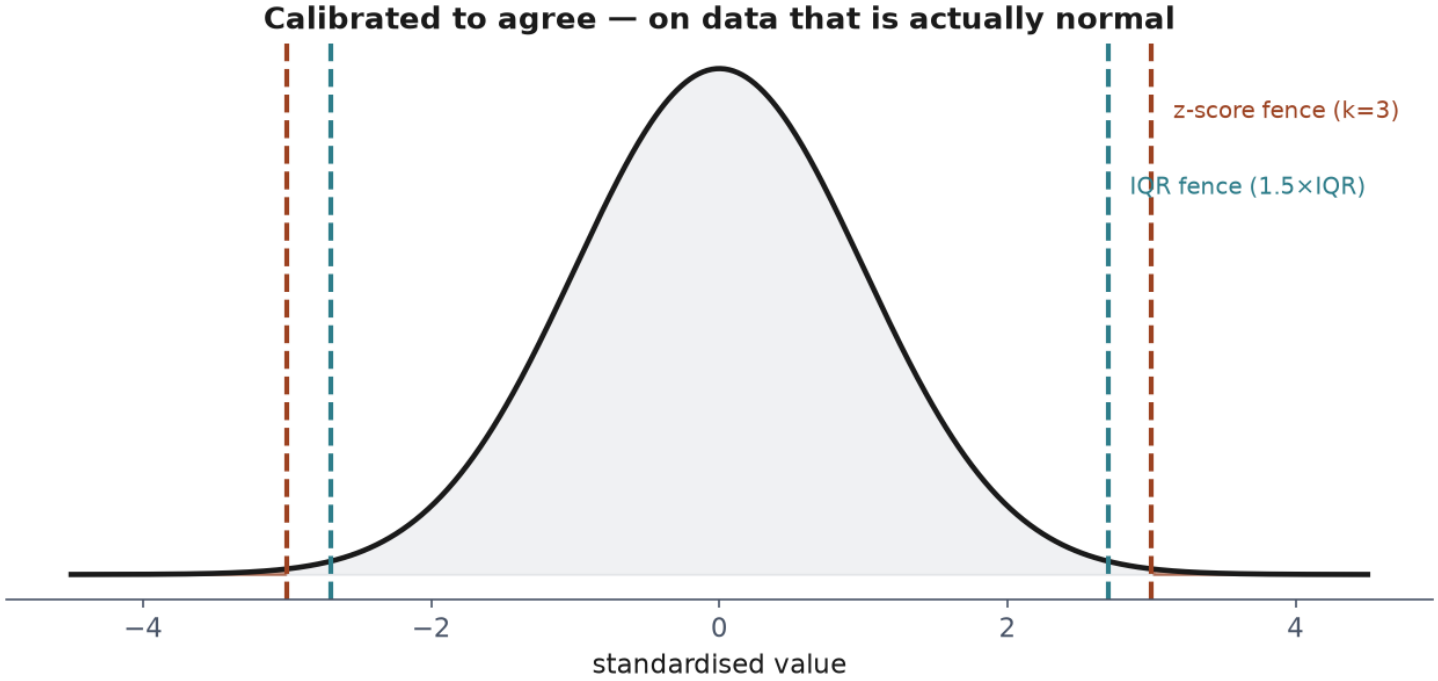
# Outliers, two rules

Two standard rules for flagging an unusual value:

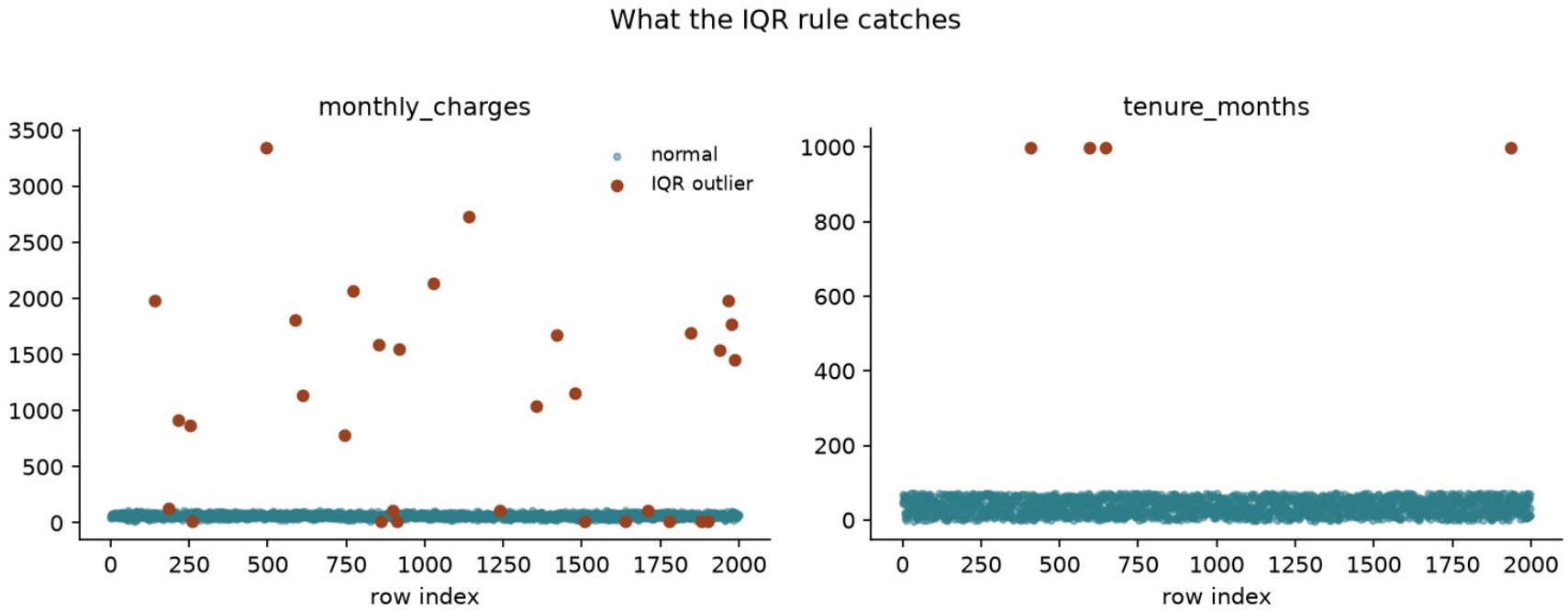
$$z_i = \frac{x_i - \bar{x}}{s}$$

$$[\ Q_1 - 1.5\ \text{IQR},\quad Q_3 + 1.5\ \text{IQR}\ ]$$

# Calibrated to agree — on a normal column



# The two rules disagree on real data




UNIVERSITÀ DEGLI STUDI DELL'AQUILA

# What neither rule can tell you

`tenure_months` of **-3** is impossible. Neither rule flags it.

- -3 is not *extreme* relative to a column spanning 0–72
- It is not *unusual*. It is *invalid* — a different question entirely

# Once something is flagged

Detection finds candidates. What to do next is a **domain** decision.

- **Cap** it at the fence (Winsorise)
- **Remove** the row
- **Correct** it, if the true value is recoverable
- **Leave** it — unusual is not the same as wrong

# Notebook 1, live

Exploration, missingness mechanisms, both outlier rules — from scratch.

# Break

# Why scale?

`tenure_months`: 0–70. `monthly_charges`: 15–150.

Gradient descent takes **one step size, along every axis, every iteration.**

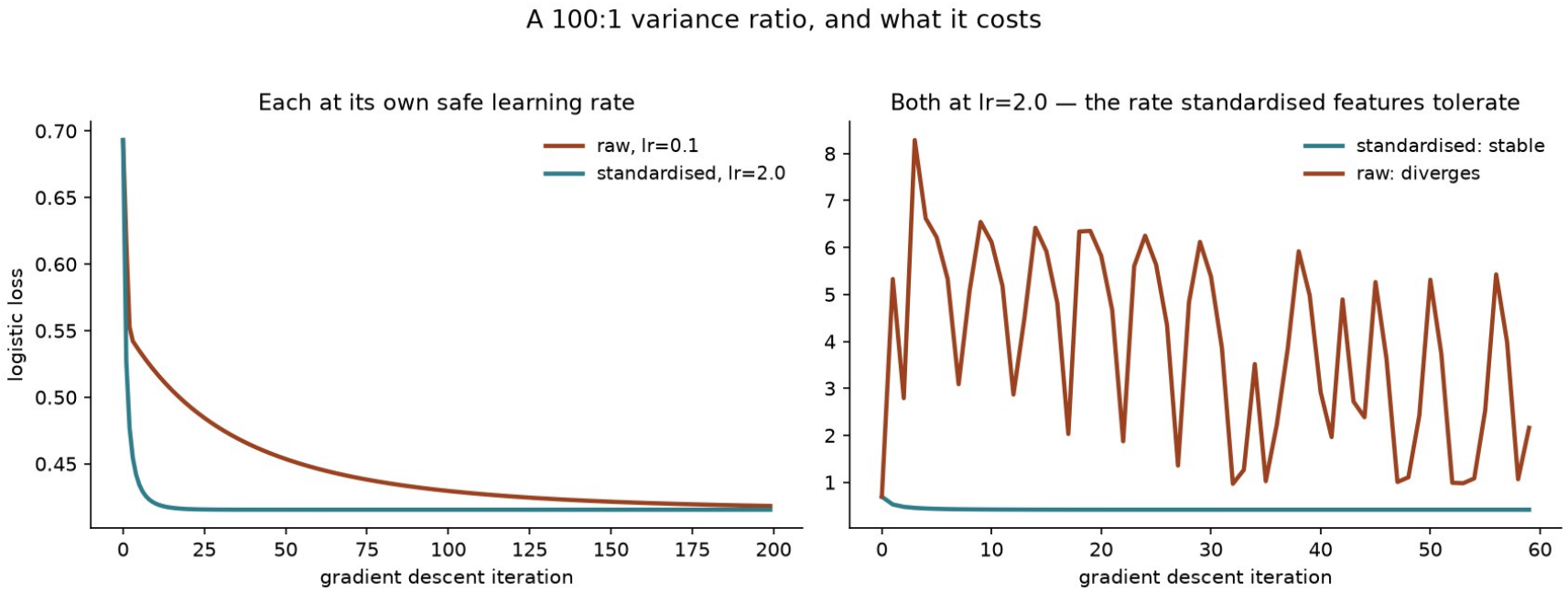
# The Hessian is the feature covariance

$$H = \nabla^2 J(w) = \frac{1}{m} X^\top X$$

For uncorrelated, centred features, H is diagonal:

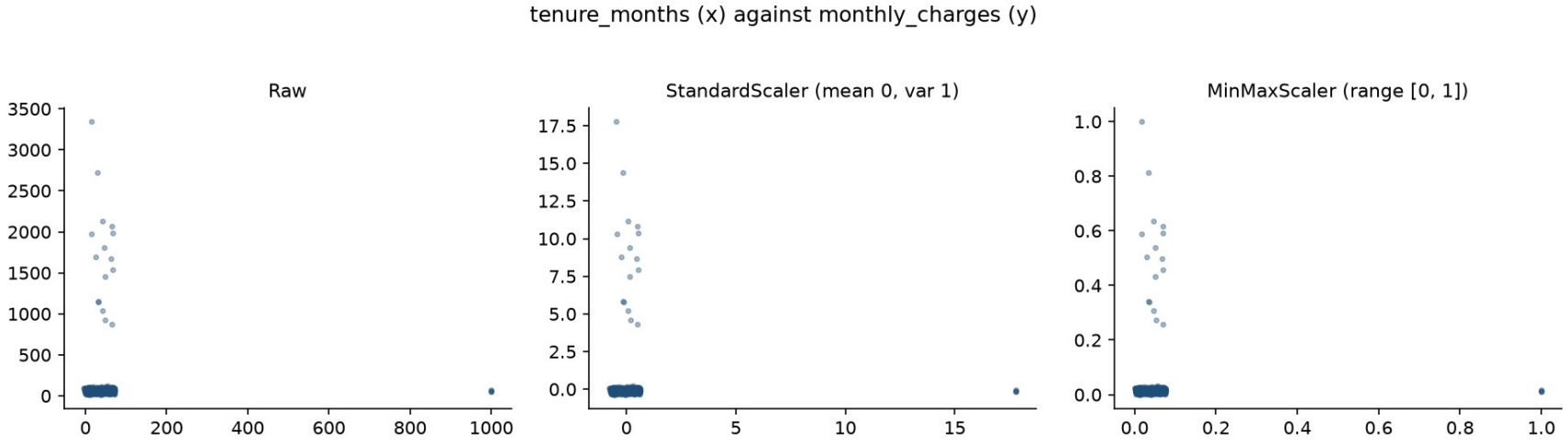
$$H = \mathrm{diag}(\sigma_1^2, \ldots, \sigma_n^2)$$

# A 100:1 variance ratio, and what it costs



# StandardScaler vs MinMaxScaler

Two standard choices, both fitted on training data only.



# The dummy variable trap

One-hot encode all k categories AND fit an intercept:

$$\sum_{j=1}^{k} \text{dummy}_j = 1 = \text{intercept}$$

# Rank deficient by exactly one

**Why all k dummies plus an intercept cannot work**

| contract_type | m2m | 1y | 2y | sum | intercept |
|---|---|---|---|---|---|
| month-to-month | **1** | 0 | 0 | **= 1** | **1** |
| one-year | 0 | **1** | 0 | **= 1** | **1** |
| two-year | 0 | 0 | **1** | **= 1** | **1** |

*the k dummy columns always sum to the intercept column — rank deficient by exactly one*

# Ordinal vs target encoding

**Ordinal**

- Categories → integers
- Only valid if a real **order** exists
- `low` < `medium` < `high`

**Target**

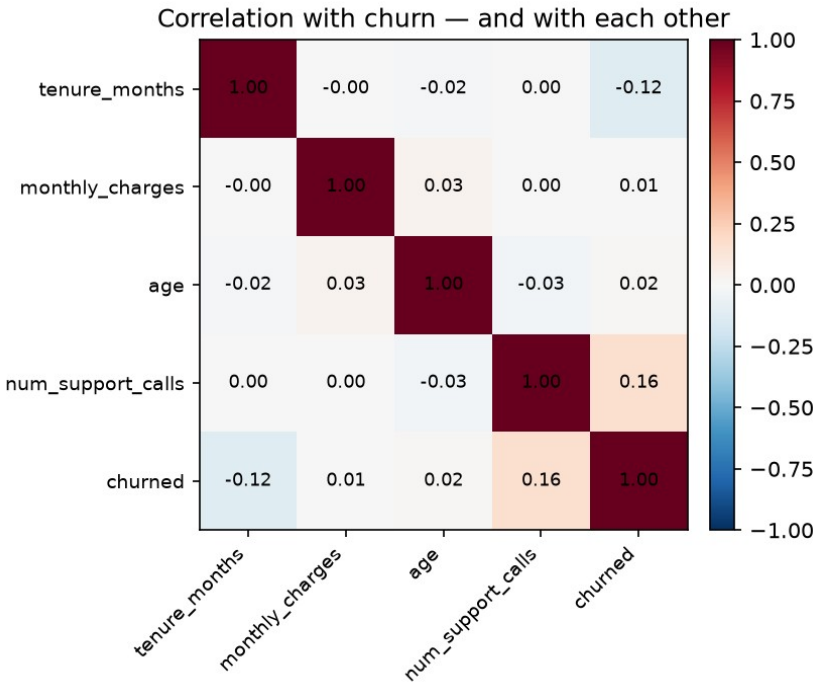
- Category → a statistic of y
- Compresses any cardinality to one column
- The dangerous one — next

# The curse of dimensionality, briefly

One-hot `zip_code`: **493** new columns, mostly zero.

- Distance-based methods (Lesson 6): everything looks equidistant
- More parameters to estimate, same sample size → more variance

# zip_code: 493 levels

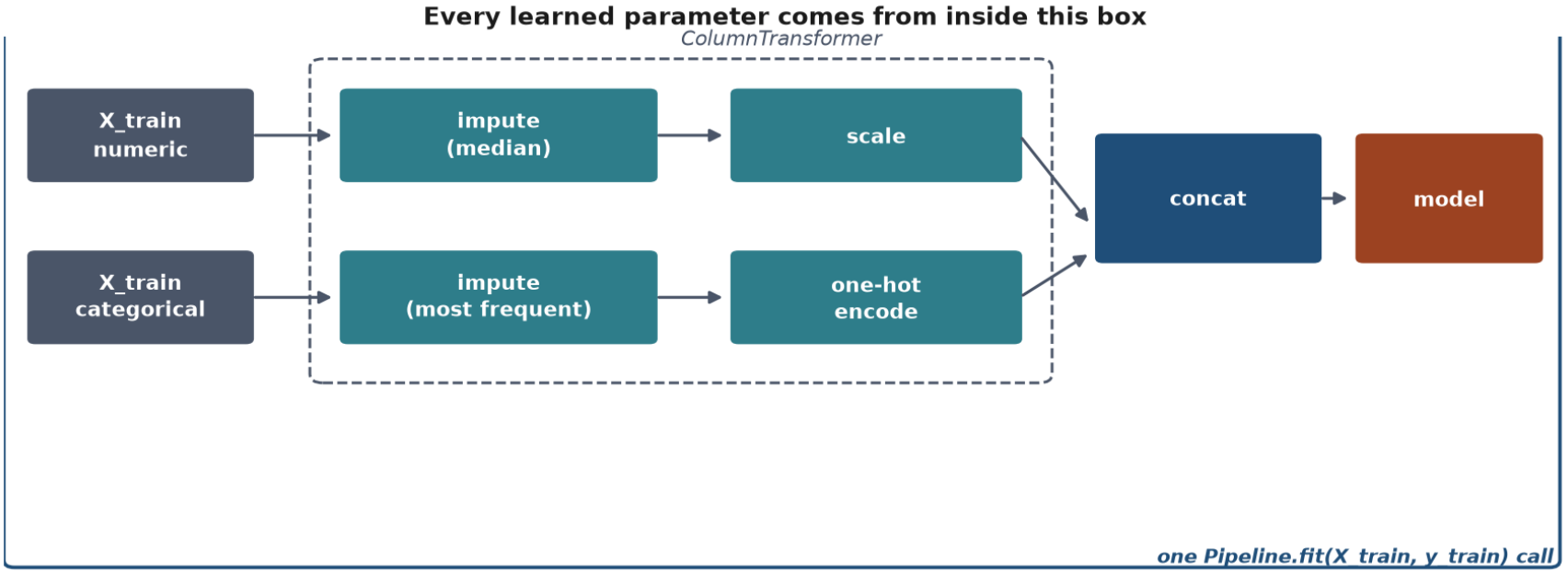


# Restating Lesson 1’s argument

f includes **every** learned parameter — not just “the model part”.

$$\mathrm{E}_{T \sim \mathcal{D}^m}\left[\hat{R}_T(f)\right] = R(f) \quad \text{only if } f \text{ is independent of } T$$

# ColumnTransformer + Pipeline



# The code

```
prep = ColumnTransformer([
    ("num", num_pipe, num_cols),
    ("cat", cat_pipe, cat_cols),
])
model = Pipeline([("prep", prep), ("clf",
clf)])
model.fit(X_train, y_train)
```

# The model, on churn

- Baseline: **0.806**
- Model (numeric + low-card categorical, no zip): **0.820** accuracy, **0.752** AUC

Modest accuracy gain. Why?

# Feature engineering

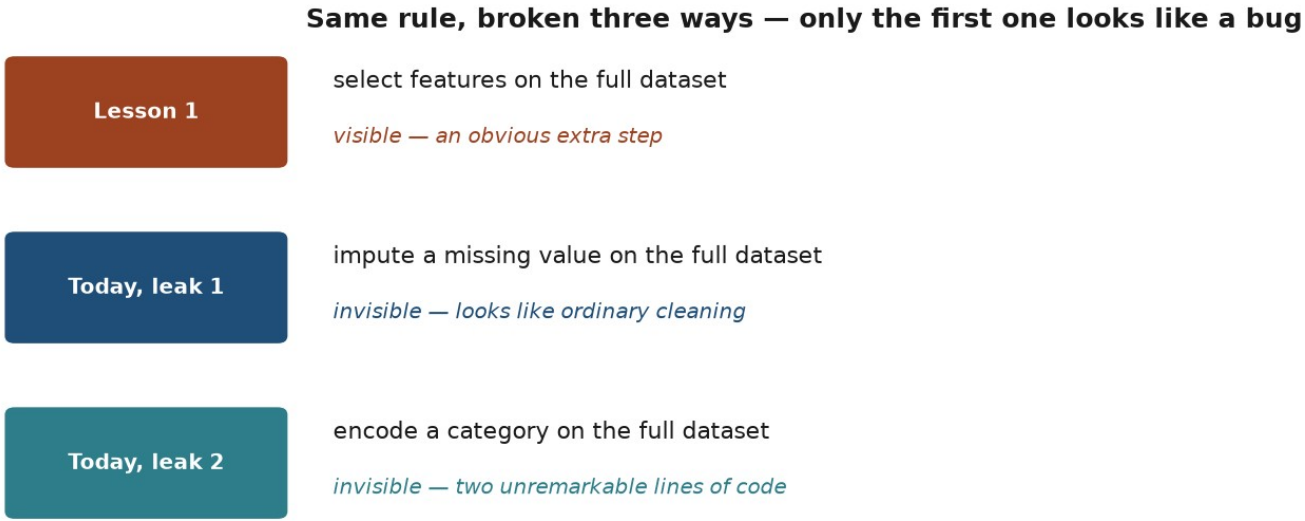
Ratios, interactions, binning — a linear model cannot invent these itself. AUC: 0.752 → 0.754, a modest but real gain.

$$\text{charge\_per\_tenure} = \frac{\text{monthly\_charges}}{\text{tenure\_months} + 1}$$

# Notebook 2, live

Scaling from scratch, the dummy trap, `ColumnTransformer`, `Pipeline`.

# Same rule, broken three ways



# Leak 1 — impute before splitting

`KNNImputer` fills a gap using **other rows' values**.

Fit it on train + test, and some of those rows are in the test set.

# The smoking gun

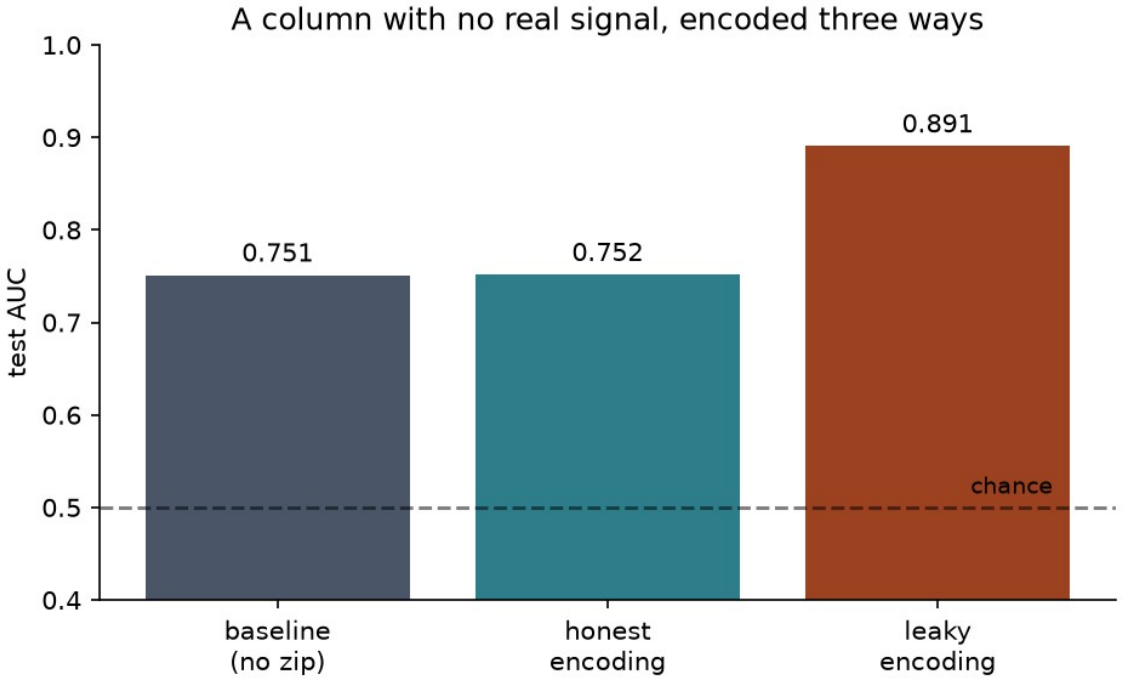
**98 of 128** training rows with missing age had a **test-set row** among their five nearest neighbours.

Nothing raised an error.

# Leak 2 — encode before splitting

Replace `zip_code` with the **average churn rate of its own customers**, computed before anybody has split anything.

# A column with no real signal, encoded three ways

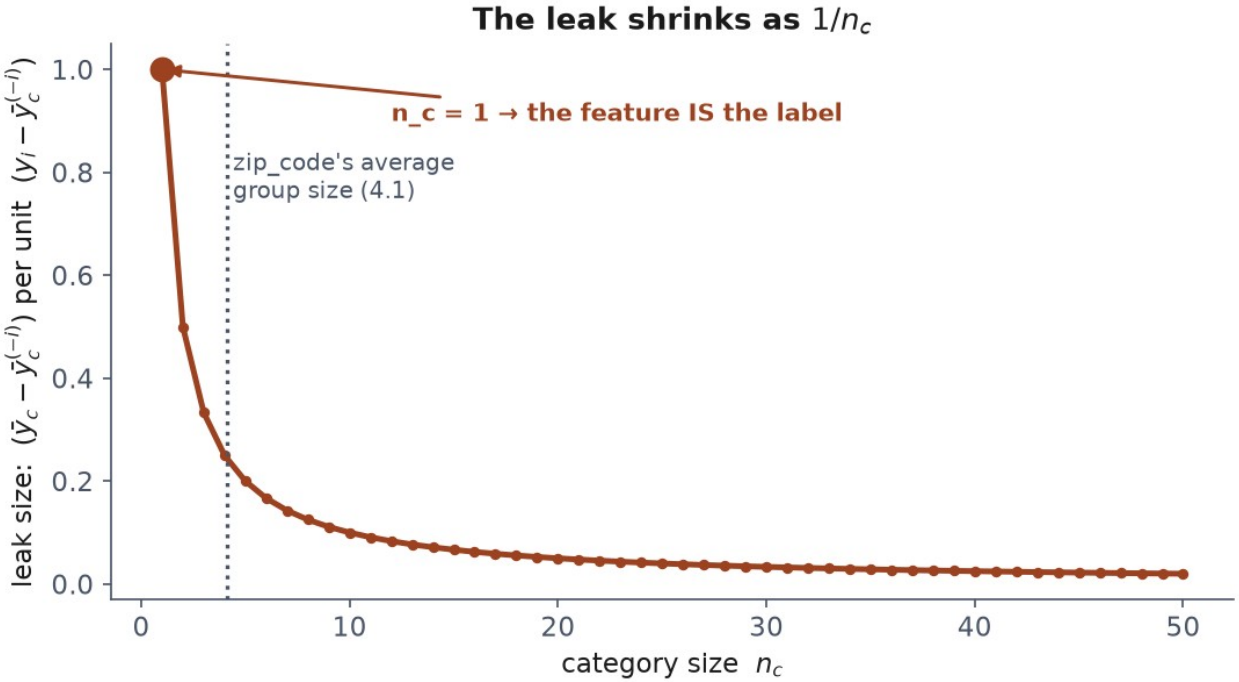


# Why small groups make it worse

The “leave-in” encoding differs from the honest leave-one-out encoding by:

$$\bar{y}_c - \bar{y}_c^{(-i)} = \frac{y_i - \bar{y}_c^{(-i)}}{n_c}$$

# The leak shrinks as 1/n_c



# The fix

`sklearn.preprocessing.TargetEncoder` — cross-fitted inside the pipeline.

Each row’s encoded value comes from **other** folds, never its own label.

# The rule, restated

Everything that **learns from data** belongs inside the fold it is fitted on.

A mean. A median. A set of neighbours. A per-category average. A set of bin edges.

# Notebook 3, live

Trace the imputation leak. Watch the encoding leak manufacture 0.89 from noise.

# What we did today

- Explored before transforming — types, gaps, shape, correlations
- Diagnosed *why* values are missing, not just *how many*
- Two outlier rules, and what neither can tell you
- Scaling, encoding, engineering — all inside `Pipeline`
- Two leaks that look nothing like leaking

# Homework — due Friday 9 October

`Exercises/02_data_exploration_and_preparation.md`

Build the full preparation pipeline yourself, and **find a leak on purpose** before fixing it.

# Before next week

- Work the three notebooks in order
- Read the handout — the derivations are examinable
- Take the quiz
- **Do the exercise**

Next: regression — the first model we derive completely.